

WaterFlow ERP

Complete Project Description

Architecture, functionality, backend modules, mobile app, admin dashboard, database design, business flows, setup, deployment, CI/CD, and production notes.

Generated from **PROJECT_DESCRIPTION.md**

Water distribution management system handover document

Contents

A quick map of the project description

1	Project Overview
2	Main Purpose
3	Technology Stack
4	High-Level Architecture
5	Backend Structure
6	Backend Application Flow
7	Backend API Routing
8	Authentication and Authorization
9	Database Design
10	Core Business Modules
11	Scheduler Jobs
12	Admin Dashboard
13	Mobile App
14	Customer Journey
15	Admin Journey
16	Driver Journey
17	Super Admin Journey
18	Main Business Flows
19	API Response Format
20	Environment Variables
21	Seed Data
22	Running the Project Locally
23	Build and Test Commands
24	CI/CD
25	Deployment Notes
26	Security and Operational Notes
27	Known Project Observations
28	File Map
29	Summary

Project Overview

Section 1

WaterFlow ERP is a full-stack water distribution management system for businesses that deliver water cans/campers to customers. The project manages the complete operational cycle: customer registration, distributor assignment, delivery scheduling, regular and extra orders, driver worklists, inventory movement, invoice generation, Razorpay payments, notifications, reporting, and admin management.

The project is organized as a monorepo with three main applications:

waterflow-erp/	
backend/	Node.js + Express + Prisma REST API
admin-dashboard/	React + Vite admin web dashboard
mobile-app/	React Native mobile app for customers, drivers, and admins
docs/	Architecture, API, database, and deployment documentation
docker-compose.yml	Local/full-stack Docker orchestration

Main Purpose

Section 2

The system is designed for water camper distribution businesses that need to:

- Maintain customers and their addresses, delivery areas, schedules, deposits, and rates.
- Assign customers to distributors/admins based on area, pincode, or GPS radius.
- Assign drivers and vehicles for daily delivery operations.
- Generate daily regular orders automatically from customer schedules.
- Allow customers to request extra water campers.
- Track delivery status and empty camper collection.
- Maintain camper inventory with audit logs.
- Generate invoices for delivered orders.
- Accept online payments through Razorpay and manual payments such as cash/UPI.
- Send notifications to customers, drivers, and admins.
- View dashboard analytics and reports.

Technology Stack

Section 3

Backend

- Node.js
- Express.js
- TypeScript
- Prisma ORM

- PostgreSQL
- Zod for request validation
- JWT access and refresh tokens
- bcryptjs for password hashing
- Razorpay for payments
- Firebase Admin SDK for push notifications
- PDFKit for invoice PDF generation
- ExcelJS for report exports
- Swagger/OpenAPI for API documentation
- Winston and Morgan for logging

Admin Dashboard

- React
- Vite
- TypeScript
- Material UI
- Redux Toolkit
- React Redux
- TanStack React Query
- Axios
- React Router
- Recharts
- Notistack

Mobile App

- React Native CLI
- TypeScript
- Redux Toolkit
- React Navigation
- Axios
- AsyncStorage
- Firebase Messaging
- Notifee
- Razorpay React Native SDK
- React Native Vector Icons

Infrastructure

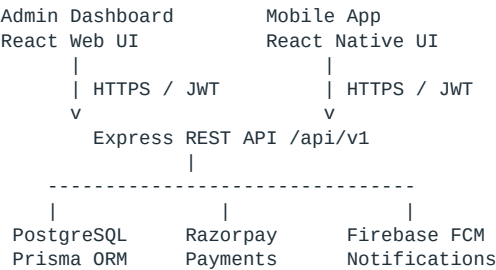
- Docker
- Docker Compose
- GitHub Actions CI/CD

- PostgreSQL container for local deployment
- Nginx container for admin dashboard production hosting

High-Level Architecture

Section 4

The system follows a three-tier architecture:



The backend is the central service. Both the admin dashboard and mobile app communicate with the backend through REST APIs. PostgreSQL stores all business data. Razorpay handles payment orders, verification, refunds, and webhooks. Firebase Cloud Messaging sends push notifications.

Backend Structure

Section 5

The backend follows a modular layered architecture:

backend/src/	
app.ts	Express app setup
server.ts	Server bootstrap
routes.ts	Central API router
config/	Environment, Prisma, Firebase, Razorpay, Swagger, logging
middlewares/	Auth, validation, error handling, rate limit, upload
utils/	JWT, password, pagination, API response helpers
jobs/	Scheduler jobs
modules/	
admin/	
auth/	
billing/	
customer/	
dashboard/	
delivery/	
distributor/	
driver/	
inventory/	
notification/	
order/	
payment/	
report/	
servicearea/	
settings/	
vehicle/	

Most backend modules use this pattern:

module.routes.ts	HTTP routes and route-level middleware
module.controller.ts	Request/response orchestration
module.service.ts	Business logic
module.repository.ts	Prisma database operations, where used
module.dto.ts	Zod validation schemas

Backend Application Flow

Section 6

Server Startup

The backend starts from `backend/src/server.ts`.

Startup flow:

- Load environment variables.
- Connect to PostgreSQL using Prisma.
- Warm up Firebase Admin SDK if configured.
- Create the Express application.
- Start the HTTP server on the configured port.
- Start in-process scheduler jobs.
- Register graceful shutdown handlers for SIGTERM and SIGINT.

Express App Setup

The Express app in `backend/src/app.ts` configures:

- Security headers with Helmet.
- CORS with configured origins.
- Response compression.
- Raw body parsing for Razorpay webhooks.
- JSON and URL-encoded request parsing.
- HTTP logging with Morgan and Winston.
- Static serving for uploaded files under `/uploads`.
- Health check endpoint at `/health`.
- Swagger docs at `/api/docs`.
- Rate limiting.
- Main API router under `/api/v1`.
- Not-found and global error handlers.

Backend API Routing

Section 7

The main router is defined in `backend/src/routes.ts`.

Shared Routes

- `/auth`
- `/notifications`

Admin-Facing Routes

- /customers
- /orders
- /inventory
- /billing
- /payments
- /deliveries
- /dashboard
- /reports
- /settings
- /vehicles
- /drivers
- /admins
- /service-areas

Customer Mobile Routes

- /me
- /me/distributors
- /me/orders
- /me/billing
- /me/payments
- /me/deliveries

Driver Mobile Routes

- /driver

Authentication and Authorization

Section 8

The project supports multiple account types:

- Super Admin
- Admin / Distributor
- Customer
- Driver

Admin Login

Admins log in with email and password:

POST /api/v1/auth/admin/login

The backend verifies the bcrypt password hash and returns:

- Access token
- Refresh token
- User profile

OTP Login

Customers, drivers, and mobile admins can log in with mobile number and OTP:

```
POST /api/v1/auth/otp/request
POST /api/v1/auth/otp/verify
```

In development or OTP test mode, the fixed OTP is **123456**.

OTP verification resolves the account type in this general order:

- Admin mobile number
- Driver mobile number
- Existing customer mobile number
- New customer auto-registration

JWT Tokens

The system uses:

- Short-lived access tokens.
- Long-lived refresh tokens.
- Refresh token rotation.
- Refresh token persistence in the database.
- Token revocation on logout.

Route Guards

Backend route protection is handled through middleware:

- `authenticate()` requires a valid JWT.
- `authenticate('admin')` restricts a route to admin principals.
- `authenticate('customer')` restricts a route to customer principals.
- `authenticate('driver')` restricts a route to driver principals.
- `authorize('SUPER_ADMIN')` restricts admin routes by role.

Database Design

Section 9

The Prisma schema is in `backend/prisma/schema.prisma`.

Main models:

- Admin
- ServiceArea
- Driver
- Vehicle

- Customer
- DeliverySkip
- CustomerSchedule
- Order
- OrderItem
- Delivery
- Inventory
- InventoryLog
- Invoice
- InvoiceItem
- Payment
- Notification
- RefreshToken
- OtpCode
- Setting

Important Enums

- AdminRole: SUPER_ADMIN, ADMIN
- DriverStatus: ACTIVE, INACTIVE
- CustomerType: DAILY, WEEKLY, MONTHLY
- CustomerStatus: ACTIVE, INACTIVE
- Weekday: Monday through Sunday
- OrderType: REGULAR, EXTRA
- OrderStatus: PENDING, ACCEPTED, PROCESSING, DELIVERED, CANCELLED
- DeliveryStatus: PENDING, DELIVERED, CANCELLED
- InventoryAction: stock and camper movement actions
- InvoiceStatus: PENDING, PARTIALLY_PAID, PAID, OVERDUE, CANCELLED
- PaymentStatus: PENDING, SUCCESS, FAILED, REFUNDED
- PaymentMethod: RAZORPAY, CASH, UPI, CARD, ADJUSTMENT
- NotificationAudience: ADMIN, CUSTOMER, DRIVER
- NotificationType: order, billing, payment, inventory, driver, and general notification types

Core Business Modules

Section 10

Auth Module

The auth module handles:

- Admin email/password login.

- OTP request and verification.
- Firebase login support.
- Current user profile lookup.
- Refresh token rotation.
- Logout and refresh token revocation.
- FCM token updates.

Admin Module

The admin module is mainly for super admins.

Functionality:

- Create admins/distributors.
- Update admin profile and service coverage.
- Delete/deactivate admins.
- List admins.
- View admin details.
- View customers assigned to an admin/distributor.

Admins can act as distributors. They may have:

- GPS latitude and longitude.
- Service radius in kilometers.
- Pincode list.
- Service area list.
- Linked master service areas.

Service Area Module

The service area module maintains a master list of areas/localities.

Functionality:

- Create service areas.
- Update service areas.
- Delete service areas.
- Assign service areas to admins.
- Store city, pincode, latitude, and longitude.

Distributor Module

The distributor module helps customers discover matching distributors.

Matching can use:

- Customer GPS coordinates.
- Pincode.
- Area/locality name.
- Fallback to all active distributors.

The mobile app uses this during customer profile completion and distributor selection.

Customer Module

The customer module manages:

- Customer CRUD.
- Customer profile.
- Mobile number.
- Address, area, landmark, and pincode.
- GPS location.
- Customer type: daily, weekly, monthly.
- Status.
- Security deposit.
- Rate per camper.
- Allocated campers.
- Assigned driver.
- Assigned distributor.
- Weekly delivery schedule.
- Delivery pause/resume.
- Skipped delivery dates.

Customer-facing routes under [/me](#) allow customers to manage their own profile, schedule, pause state, and skip dates.

Driver Module

The driver module manages:

- Driver CRUD.
- Driver mobile login through OTP.
- Driver status.
- Driver zone.
- Driver vehicle assignment.
- Customer assignment to drivers.
- Driver notifications.
- Driver mobile dashboard.
- Driver delivery worklist.
- Driver FCM token updates.

The driver-facing mobile routes allow drivers to:

- View their profile.
- View assigned customers.
- View today's deliveries.
- Mark deliveries as delivered or cancelled.
- View driver notifications.

Vehicle Module

The vehicle module manages:

- Vehicle number.
- Vehicle type.
- Capacity.
- Active/inactive state.
- Notes.
- One-to-one assignment with a driver.

Order Module

Orders represent customer demand for water campers.

Order types:

- **REGULAR**: generated from customer schedule.
- **EXTRA**: requested manually by customer/admin.

Order functionality:

- Admin can create orders.
- Customers can request extra orders.
- Admin can list and filter orders.
- Order status can be updated.
- Regular orders are generated automatically by the scheduler.
- Orders have line items with quantity, rate, and amount.

Delivery Module

Deliveries are connected to orders.

Delivery functionality:

- List deliveries.
- Mark delivery status.
- Record delivered quantity.
- Record empty campers collected.
- Assign driver.
- Cancel delivery.
- Update inventory during delivery operations where business logic requires it.

Inventory Module

The inventory module tracks camper stock.

The **Inventory** table is a single-row snapshot with id **default**.

Tracked counts:

- Total campers
- Filled campers

- Empty campers
- Damaged campers
- Lost campers
- Returned campers
- Allocated campers

Every inventory movement creates an [InventoryLog](#) row.

Inventory actions:

- [STOCK_IN](#)
- [FILLED](#)
- [EMPTIED](#)
- [ALLOCATED](#)
- [RETURNED](#)
- [DAMAGED](#)
- [LOST](#)
- [ADJUSTMENT](#)

Low inventory can trigger admin notifications.

Billing Module

The billing module handles invoices.

Functionality:

- Generate invoice for a customer and period.
- Auto-generate invoices by customer plan/type.
- Aggregate delivered orders in the billing period.
- Calculate quantity, rate, subtotal, tax, total, paid amount, and due amount.
- Generate invoice PDF.
- Notify customers when bills are generated.
- Track invoice status.
- Apply payments to invoices.
- Mark overdue invoices.
- Send due reminders.

Invoice statuses:

- [PENDING](#)
- [PARTIALLY_PAID](#)
- [PAID](#)
- [OVERDUE](#)
- [CANCELLED](#)

Payment Module

The payment module supports online and manual payments.

Razorpay flow:

- Customer requests a Razorpay order from backend.
- Backend creates Razorpay order.
- Mobile app opens Razorpay checkout.
- Customer completes payment.
- Mobile app sends payment id, order id, and signature to backend.
- Backend verifies signature.
- Payment is marked successful.
- Billing module applies the payment to invoice.

Manual payment flow:

- Admin records cash, UPI, card, or adjustment payment.
- Payment is linked to customer and optionally invoice.
- Invoice paid/due amount is updated.

Webhook:

- Razorpay webhook endpoint receives server-to-server confirmation.
- Raw request body is used for signature validation.

Notification Module

Notifications are stored in the database and can be pushed through FCM.

Audiences:

- Admin
- Customer
- Driver

Examples:

- New customer.
- New order request.
- Order accepted.
- Order delivered.
- Bill generated.
- Payment success.
- Payment failed.
- Payment due reminder.
- Inventory low.
- Driver assigned.
- Delivery assigned.
- Customer skipped delivery.
- General notification.

Dashboard Module

The dashboard module provides summary analytics.

Typical dashboard data:

- Customer counts.
- Order counts.
- Delivery counts.
- Revenue summary.
- Inventory snapshot.
- Payment status.
- Chart data for admin dashboard.

Report Module

The report module provides report data and export functionality.

Supported report types include:

- Daily
- Weekly
- Monthly
- Yearly
- Revenue
- Customer
- Inventory
- Order
- Payment

Export formats:

- Excel
- CSV
- PDF

Settings Module

The settings module stores configurable business settings as JSON.

Examples:

- Business name, GSTIN, phone, email, address.
- Billing rate, tax percent, due days, currency.
- Inventory low threshold.

Scheduler Jobs

Section 11

The backend includes an in-process scheduler in [backend/src/jobs/scheduler.ts](#).

The scheduler can be disabled with:

```
DISABLE_SCHEDULERS=true
```

Regular Order Generation

Shortly after server boot and then hourly, the scheduler calls:

```
orderService.generateRegularOrdersForDate(new Date())
```

It creates regular orders for active customers whose delivery schedule is enabled for the current weekday. The process is intended to be idempotent so repeated scheduler runs do not duplicate orders.

Due Payment Reminders

Shortly after server boot and then hourly, the scheduler calls:

```
billingService.sendDueReminders()
```

It flags overdue invoices and sends payment reminder notifications.

Admin Dashboard

Section 12

The admin dashboard is in [admin-dashboard/](#).

Important files:

- [src/App.tsx](#): route structure and role-based routing.
- [src/api/client.ts](#): Axios client with auth token and refresh interceptor.
- [src/api/endpoints.ts](#): typed API functions.
- [src/features/auth/authSlice.ts](#): auth state.
- [src/components/Layout.tsx](#): dashboard layout.
- [src/pages/*](#): dashboard screens.

Admin Dashboard Pages

Super admin pages:

- Admins
- Admin details
- Service areas
- Profile

Operational admin pages:

- Dashboard
- Customers
- Customer details
- Customer form
- Drivers
- Driver details
- Driver form

- Vehicles
- Orders
- Inventory
- Billing
- Payments
- Notifications
- Reports
- Settings
- Profile

Role-Based Routing

The dashboard uses React Router guards:

- Users without token go to login.
- Super admins can access admin management and service areas.
- Super admins are redirected away from operational pages.
- Regular admins can access operational pages.

API Handling

The admin dashboard Axios client:

- Adds `Authorization: Bearer <token>` to requests.
- Refreshes access tokens on 401 responses.
- Uses a single-flight refresh guard to avoid multiple refresh calls at once.
- Logs out the user if refresh fails.

Mobile App

Section 13

The mobile app is in `mobile-app/`.

It supports three experiences:

- Customer app
- Driver app
- Admin mobile app

Important files:

- `src/navigation/RootNavigator.tsx`: top-level navigation.
- `src/navigation/DriverNavigator.tsx`: driver navigation.
- `src/admin/navigation/AdminNavigator.tsx`: admin mobile navigation.
- `src/api/client.ts`: Axios client with refresh interceptor.
- `src/api/endpoints.ts`: customer and driver API functions.
- `src/admin/api.ts`: admin mobile API functions.

- `src/store/slices/authSlice.ts`: mobile auth state.
- `src/config.ts`: backend API URL and Razorpay public key id.

Mobile Auth Flow

- App boots and checks stored tokens.
- If no token exists, user sees OTP login.
- User enters mobile number.
- User verifies OTP.
- Backend returns account type and tokens.
- App routes user based on role:
- Admin -> Admin mobile area
- Driver -> Driver area
- Customer -> Customer app
- Customer without complete profile goes to profile completion.

Customer Screens

- Splash
- OTP login
- OTP verify
- Complete profile
- Home
- Deliveries
- Order camper
- Order history
- Billing
- Payment history
- Notifications
- Profile
- Support
- Skip deliveries

Driver Screens

- Driver deliveries
- Driver customers
- Driver notifications
- Driver profile

Admin Mobile Screens

- Dashboard
- Customers
- Customer details

- Customer form
- Drivers
- Driver details
- Driver form
- Vehicles
- Orders
- Inventory
- Billing
- Payments
- Notifications
- Reports
- Settings
- Profile
- More

Mobile API Handling

The mobile Axios client:

- Uses the configured backend base URL.
- Adds bearer token to requests.
- Refreshes tokens on 401.
- Stores tokens in AsyncStorage.
- Clears tokens and logs out when refresh fails.

Customer Journey

Section 14

Typical customer flow:

- Customer opens mobile app.
- Customer logs in using mobile number and OTP.
- If new, customer completes profile.
- Customer may select distributor based on GPS, pincode, or area.
- Customer views home dashboard.
- Customer checks delivery schedule and delivery history.
- Customer can request extra camper orders.
- Customer can pause deliveries or select skipped dates.
- Customer receives invoices.
- Customer pays online through Razorpay.
- Customer receives payment and delivery notifications.

Admin Journey

Section 15

Typical admin flow:

- Admin logs into dashboard using email/password or mobile OTP.
- Admin views dashboard KPIs.
- Admin manages customers.
- Admin configures schedules, rates, deposits, and assigned distributor/driver.
- Admin manages drivers and vehicles.
- Admin tracks orders and deliveries.
- Admin adjusts inventory.
- Admin generates invoices.
- Admin records manual payments or reviews Razorpay payments.
- Admin exports reports.
- Admin sends or reviews notifications.

Driver Journey

Section 16

Typical driver flow:

- Driver logs in using mobile number and OTP.
- Driver views assigned customers.
- Driver views daily delivery worklist.
- Driver delivers water campers.
- Driver marks delivery as delivered or cancelled.
- Driver records delivered quantity and empty campers collected.
- Driver receives delivery/customer notifications.

Super Admin Journey

Section 17

Typical super admin flow:

- Super admin logs into the admin dashboard.
- Super admin manages admins/distributors.
- Super admin creates service areas.
- Super admin assigns service areas and coverage details to admins.

- Super admin can view customers attached to each admin/distributor.

Main Business Flows

Section 18

Regular Delivery Flow

```
Customer schedule configured
|
Scheduler checks today's weekday
|
Regular order generated
|
Delivery created / visible to admin and driver
|
Driver delivers camper
|
Delivery marked delivered
|
Order status updated
|
Invoice can include delivered order
```

Extra Order Flow

```
Customer requests extra camper
|
Order created as EXTRA
|
Admin/driver sees order
|
Delivery fulfilled
|
Billing includes delivered quantity
```

Billing Flow

```
Delivered orders in date range
|
Admin generates invoice
|
System calculates amount
|
PDF invoice generated
|
Customer notified
|
Customer pays online or admin records manual payment
|
Invoice status updated
```

Payment Flow

```
Customer selects payment
|
Backend creates Razorpay order
|
Mobile app opens Razorpay checkout
|
Payment completed
|
Backend verifies signature
|
Payment marked SUCCESS
|
Invoice paid/due values updated
```

Inventory Flow

Admin adjusts inventory or delivery changes stock
|
Inventory snapshot updated
|
Inventory log created
|
Low-stock threshold checked
|
Admin notification created if needed

API Response Format

Section 19

The backend generally returns a consistent response shape:

```
{
  "success": true,
  "message": "Operation completed",
  "data": {},
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 0,
    "totalPages": 1
  }
}
```

Errors are handled by global middleware and returned with meaningful messages.

Environment Variables

Section 20

Common backend variables:

```
NODE_ENV
PORT
API_PREFIX
CORS_ORIGINS
DATABASE_URL
JWT_ACCESS_SECRET
JWT_REFRESH_SECRET
JWT_ACCESS_EXPIRES_IN
JWT_REFRESH_EXPIRES_IN
OTP_DEV_CODE
OTP_TEST_MODE
OTP_EXPIRES_MINUTES
SMS_PROVIDER
MSG91_AUTH_KEY
MSG91_TEMPLATE_ID
FAST2SMS_API_KEY
FAST2SMS_MESSAGE_ID
TWILIO_ACCOUNT_SID
TWILIO_AUTH_TOKEN
TWILIO_FROM
RAZORPAY_KEY_ID
RAZORPAY_KEY_SECRET
RAZORPAY_WEBHOOK_SECRET
FIREBASE_SERVICE_ACCOUNT_PATH
FIREBASE_SERVICE_ACCOUNT
UPLOAD_DIR
MAX_UPLOAD_MB
INVENTORY_LOW_THRESHOLD
RATE_LIMIT_WINDOW_MS
```

```
RATE_LIMIT_MAX
DISABLE_SCHEDULERS
```

Admin dashboard variable:

```
VITE_API_URL
```

Mobile app currently uses [mobile-app/src/config.ts](#) for:

```
apiUrl
razorpayKeyId
```

Seed Data

Section 21

The seed script is in [backend/prisma/seed.ts](#).

It creates:

- Super admin:
- Email: [superadmin@waterflow.com](#)
- Password: [Admin@123](#)
- Admin:
- Email: [admin@waterflow.com](#)
- Password: [Admin@123](#)
- Default inventory snapshot.
- Default business, billing, and inventory settings.
- Sample customers.
- Sample vehicle.
- Sample driver.

Development OTP:

123456

Running the Project Locally

Section 22

Backend

```
cd backend
cp .env.example .env
npm install
npm run prisma:generate
npm run prisma:migrate
npm run seed
npm run dev
```

Backend runs at:

<http://localhost:4000>

Swagger docs:

`http://localhost:4000/api/docs`

Admin Dashboard

```
cd admin-dashboard
cp .env.example .env
npm install
npm run dev
```

Admin dashboard runs at:

`http://localhost:5173`

Mobile App

```
cd mobile-app
npm install
npm run android
```

or:

```
npm run ios
```

Metro runs on port **9090** according to the mobile scripts.

Docker

From the project root:

```
docker compose up --build
```

Docker services:

- **db**: PostgreSQL 16 Alpine
- **backend**: Express API
- **admin**: Nginx serving built admin dashboard

Build and Test Commands

Section 23

Backend

```
npm run build
npm start
npm run lint
npm run format
npm test
npm run prisma:generate
npm run prisma:migrate
npm run prisma:deploy
npm run prisma:studio
npm run seed
```

Admin Dashboard

```
npm run dev
npm run build
npm run preview
npm run lint
```

Mobile App

```
npm run start
npm run android
npm run ios
npm run lint
npm test
npx tsc --noEmit
```

CI/CD

Section 24

GitHub Actions workflows are in [.github/workflows/](#).

CI

The CI workflow:

- Installs backend dependencies.
- Generates Prisma client.
- Runs Prisma migrations.
- Builds backend.
- Runs backend tests if present.
- Installs admin dashboard dependencies.
- Builds admin dashboard.
- Installs mobile app dependencies.
- Runs TypeScript check for mobile app.

Deploy

The deploy workflow:

- Builds Docker images for backend and admin dashboard.
- Pushes images to GitHub Container Registry.
- Contains a commented example for SSH deployment.

Deployment Notes

Section 25

The project includes:

- [docker-compose.yml](#) for local or server deployment.
- [backend/Dockerfile](#).
- [admin-dashboard/Dockerfile](#).
- [admin-dashboard/nginx.conf](#).
- [backend/ecosystem.config.js](#) for PM2-style deployment.
- HTML deployment guide at [WaterFlow-Deployment-Guide.html](#).

Production deployment should configure:

- Real PostgreSQL database.
- Strong JWT secrets.
- Correct CORS origins.
- Razorpay credentials.
- Razorpay webhook secret.
- Firebase service account.
- SMS provider credentials if real OTP delivery is needed.
- Object storage for invoices/uploads if scaling beyond local disk.
- External scheduler or worker for high-scale order generation.

Security and Operational Notes

Section 26

Important items to handle carefully:

- Do not commit real `.env` files.
- Do not commit private keys, Firebase service account JSON files, release keystores, or payment credential CSV files.
- Rotate any credentials that were accidentally committed or shared.
- Use strong `JWT_ACCESS_SECRET` and `JWT_REFRESH_SECRET` in production.
- Restrict CORS origins in production.
- Keep Razorpay secret keys only on the backend.
- Keep the mobile and web apps using public keys only.
- Configure HTTPS in production.
- Use production-ready storage for uploads and invoice PDFs.
- Consider Redis or another persistent store for distributed rate limiting.
- Replace the in-process scheduler with a dedicated worker or external cron when running multiple backend instances.

Known Project Observations

Section 27

Based on the current workspace:

- The project folder has `.github` and `.gitignore`, but the folder itself is not currently initialized as a Git repository.
- Build artifacts exist in the workspace, including backend `dist`, admin `dist`, mobile Android build output, and mobile `node_modules`.
- Some documentation appears older than the current code. For example, the mobile README mentions an older React Native version, while `mobile-app/package.json` uses React Native `0.83.1`.
- The admin dashboard and mobile app currently contain fallback API URLs pointing to a live IP address. For clean deployment, these should be controlled through environment configuration.

- Sensitive-looking files are present in the workspace and should be reviewed before publishing or sharing the codebase.

File Map

Section 28

Important files and folders:

```
README.md
PROJECT_DESCRIPTION.md
docker-compose.yml
docs/
  ARCHITECTURE.md
  API.md
  DEPLOYMENT.md
  database.sql
backend/
  package.json
  Dockerfile
  ecosystem.config.js
  prisma/
    schema.prisma
    seed.ts
src/
  server.ts
  app.ts
  routes.ts
  config/
  middlewares/
  utils/
  jobs/
  modules/
admin-dashboard/
  package.json
  Dockerfile
  nginx.conf
  vite.config.ts
src/
  App.tsx
  main.tsx
  api/
  app/
  components/
  features/
  pages/
  theme/
  types/
mobile-app/
  package.json
  App.js
  index.js
  android/
  ios/
src/
  config.ts
  api/
  admin/
  components/
  dashboard/
  navigation/
  screens/
  services/
  store/
  theme/
```

Summary

Section 29

WaterFlow ERP is a complete operational platform for a water camper distribution business. The backend provides a structured REST API with Prisma/PostgreSQL, authentication, scheduling, billing, inventory, payments, notifications, reports, and role-based access. The web dashboard gives admins and super admins a full management interface. The React Native mobile app gives customers, drivers, and mobile admins role-specific workflows.

The most important technical strengths are:

- Clear modular backend structure.
- Typed frontend API layers.
- Multi-role authentication.
- Prisma schema covering the full business domain.
- Built-in billing, payments, notifications, and reports.
- Docker and CI/CD setup.

The most important cleanup tasks before production or public sharing are:

- Remove or secure secrets and credential files.
- Remove generated build artifacts from source control.
- Update stale documentation.
- Move hardcoded API URLs into environment-based configuration.
- Use stronger production infrastructure for scheduling, uploads, and rate limiting.